

Język Python

Wykład 04

Fizyka Medyczna

27 października 2026

Funkcje i obsługa plików

Jacek Gatlik

e-mail: jacek.gatlik@agh.edu.pl
www: https://galaxy.agh.edu.pl/~jgatlik/

Wydział Fizyki i Informatyki Stosowanej,
Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

Plan wykładu

Funkcje, ciąg dalszy

Podstawy obsługi plików

Przykłady

Zadanie dodatkowe

Sekcja 1

Funkcje, ciąg dalszy

Obsługa błędów

Wyjątki są mechanizmem sygnalizowania błędów w Pythonie. Gdy w programie wystąpi błąd, zostaje podniesiony wyjątek, który może być przechwycony i obsłużony.

Obsługa wyjątków

Do obsługi wyjątków służy konstrukcja `try-except`:

```
try:
    pass  # kod, który może podnieść wyjątek
except ValueError as e:
    pass  # obsługa konkretnego typu wyjątku
```

Przykład:

```
try:
    wynik = 10 / 0
except ZeroDivisionError as e:
    print(f"Błąd: {e}")
```

Obsługa błędów: else i finally

Bloki *else* i *finally*

- `else`: Wykonywane po normalnym zakończeniu bloku `try`, bez wyjątku ani wcześniejszego wyjścia (np. `return`).
- `finally`: Wykonywane zawsze, niezależnie od tego, czy wyjątek wystąpił.

```python
try:
    wynik = 10 / 2
except ZeroDivisionError as e:
    print(f"Błąd: {e}")
else:
    print(f"Wynik: {wynik}")
finally:
    print("Koniec operacji.")
```

Zmienna lokalna i globalna

- **Zmienna lokalna**: Zmienna zdefiniowana wewnątrz funkcji, dostępna tylko w tej funkcji.
- **Zmienna globalna**: Zmienna zdefiniowana na najwyższym poziomie modułu, dostępna w całym module.

```python
x = 10  # zmienna globalna
def funkcja():
    x = 5  # zmienna lokalna
    print(f"Wewnątrz funkcji x = {x}")

funkcja()
print(f"Poza funkcją x = {x}")
```

```
Wewnątrz funkcji x = 5
Poza funkcją x = 10
```

Zmienna lokalna i globalna

Zasada LEGB: Python stosuje zasadę LEGB (Local, Enclosing, Global, Built-in) przy wyszukiwaniu nazw zmiennych.

- Local: Zmienne lokalne w bieżącej funkcji.
- Enclosing: Zmienne w funkcjach zewnętrznych (dla funkcji zagnieżdżonych).
- Global: Zmienne globalne w module.
- Built-in: Wbudowane nazwy Pythona.

Słowo kluczowe *global*

Aby przypisać nową wartość nazwie globalnej wewnątrz funkcji, używamy `global`. Sam odczyt tej nazwy lub modyfikacja wskazywanej listy nie wymaga `global`.

```
x = 10

def funkcja():
    global x
    x = 5

funkcja()
print(f"x = {x}")  # Wyświetli: x = 5
```

Zmienna lokalna i globalna

Słowo kluczowe *nonlocal*

`nonlocal` pozwala przypisać nową wartość nazwie z najbliższej otaczającej funkcji, w której ta nazwa już istnieje. Nie odnosi się do nazw globalnych.

```
def zewnetrzna():
    x = 10
    def wewnetrzna():
        nonlocal x
        x = 5
    wewnetrzna()
    print(f"x = {x}")

zewnetrzna()  # Wyświetli: x = 5
```

Nazwy, wpisy dokumentacyjne i wskazówki dotyczące typów

Nazwy funkcji
Nazwy funkcji w Pythonie powinny być wybierane z myślą o czytelności i zgodności z konwencjami nazewnictwa. Zgodnie z wytycznymi PEP 8, nazwy funkcji powinny:

- Być pisane małymi literami.
- Używać podkreśleń (_) do oddzielania słów w nazwie (konwencja *snake_case*).

```
1 def oblicz_sume(a, b):
2     return a + b
```

Dobrze dobrana nazwa funkcji ułatwia zrozumienie jej celu bez konieczności analizowania ciała funkcji.
Wpisy dokumentacyjne (docstrings)
Wpisy dokumentacyjne to ciągi znaków umieszczone zaraz po definicji funkcji, które służą do dokumentowania jej działania. Są one dostępne w czasie wykonywania programu poprzez atrybut `__doc__` lub wywołanie `help(funkcja)`.

```
1 def nazwa_funkcji(parametry):
2     """Opis funkcji."""
3     pass  # miejsce na ciało funkcji
```

Nazwy, wpisy dokumentacyjne i wskazówki dotyczące typów

```python
def policz_pole_kola(r):
    """Oblicza pole koła o promieniu r.

    Parametry:
        r (float): Promień koła.

    Zwraca:
        float: Pole koła.
    """
    import math
    return math.pi * r ** 2
```

Nazwy, wpisy dokumentacyjne i wskazówki dotyczące typów

Przykład stylu Google:

```python
def oblicz_kwadrat(n):
    """Oblicza kwadrat liczby.

    Args:
        n (int or float): Liczba do podniesienia do kwadratu.

    Returns:
        int or float: Kwadrat liczby n.
    """
    return n ** 2
```

Dokumentacja i wskazówki dotyczące typów

Wskazówki dotyczące typów (type hints)
W Pythonie od wersji 3.5 wprowadzono opcjonalne wskazówki dotyczące typów, które pozwalają na określenie oczekiwanych typów parametrów i wartości zwracanych przez funkcje. Nie są one wymuszane podczas wykonywania programu, ale ułatwiają statyczną analizę kodu za pomocą narzędzi takich jak mypy.

Składnia:

```
def nazwa_funkcji(parametr: float) -> float:
    pass  # miejsce na ciało funkcji
```

Przykład:

```
def pole_prostokata(dlugosc: float, szerokosc: float) -> float:
    """Zwraca pole prostokąta dla podanych długości boków."""
    return dlugosc * szerokosc
```

Rekurencja

Rekurencja to technika programowania, w której funkcja wywołuje samą siebie. Jest często używana do rozwiązywania problemów, które można podzielić na mniejsze, podobne podproblemy.

Warunki rekurencji

- **Warunek bazowy**: Definiuje przypadek, w którym rekurencja zostaje zatrzymana.
- **Wywołanie rekurencyjne**: Funkcja wywołuje samą siebie z argumentem przybliżającym do przypadku bazowego.

Przykład: Ciąg Fibonacciego (całkowite `n >= 0`)

```python
def fib(n):
    if n <= 1:
        return n
    else:
        return fib(n - 1) + fib(n - 2)

print(fib(6))  # Wyświetli: 8
```

Ta prosta wersja wielokrotnie liczy te same podproblemy; liczba wywołań rośnie wykładniczo.

Rekurencja a iteracja: silnia

Zakładamy, że `n` jest liczbą całkowitą i `n >= 0`.

Rekurencja

```python
def silnia_rekurencja(n):
    if n <= 1:
        return 1
    return n * silnia_rekurencja(n - 1)

print(silnia_rekurencja(10))
```

Iteracja

```python
def silnia_iteracja(n):
    wynik = 1
    for i in range(2, n + 1):
        wynik *= i
    return wynik

print(silnia_iteracja(10))
```

Obie wersje zwracają `3628800`. Rekurencja musi zachować stan każdego niezakończonego wywołania.

Rekurencja a iteracja: pomiar czasu

Pojedynczy pomiar krótkiej operacji jest podatny na zakłócenia. Moduł `timeit` pozwala powtórzyć obliczenia.

```python
from timeit import repeat

# Funkcje silni z poprzedniego slajdu.
n = 100
powtorzenia = 1000
for funkcja in (silnia_rekurencja, silnia_iteracja):
    czasy = repeat(lambda: funkcja(n),
                   number=powtorzenia, repeat=5)
    czas_us = min(czasy) / powtorzenia * 1e6
    print(f"{funkcja.__name__}: {czas_us:.2f} us")
```

Interpretacja: Pokazujemy najmniejszy średni czas z pięciu serii. Wyniki zależą od komputera, wersji Pythona i danych; nie dowodzą ogólnej przewagi jednego algorytmu.

Rekurencja a iteracja: porównanie

Kryterium	Rekurencja	Iteracja
Definicja	Funkcja wywołuje samą siebie.	Instrukcje powtarzane w pętli.
Zakończenie	Osiągnięcie przypadku bazowego.	Warunek pętli, wyczerpanie iteratora lub `break`.
Typowe użycie	Drzewa, fraktale, algorytmy dziel i zwyciężaj.	Sekwencyjne przetwarzanie danych.
Liczba kroków dla silni	Liniowa względem `n`.	Liniowa względem `n`.
Pamięć dla silni	Stos $O(n)$ aktywnych wywołań.	Stała liczba zmiennych pomocniczych.

Złożoność zależy od algorytmu. W porównaniu pamięci pomijamy rozmiar liczb całkowitych, który rośnie wraz z wynikiem silni.

Rekurencja a iteracja: zalety i ograniczenia

	Rekurencja	**Iteracja**
Zalety	Naturalny zapis problemów o budowie hierarchicznej; często krótki kod.	Brak narastania stosu wywołań; często mniejszy narzut w prostych obliczeniach.
Ograniczenia	Koszt wywołań funkcji i limit głębokości rekurencji.	Dla drzew może wymagać jawnej listy pełniącej rolę stosu.
Typowe błędy	Brak przypadku bazowego lub brak zbliżania się do niego.	Błędne indeksy lub warunek zakończenia pętli.

Bieżący limit odczytamy przez `sys.getrecursionlimit()`. W środowisku Jupyter może być inny niż w samodzielnym skrypcie.

Limit rekurencji i stos wywołań

Każde wywołanie czeka na wynik kolejnego i zachowuje swój stan:

$$\texttt{silnia(3)} \longrightarrow \texttt{silnia(2)} \longrightarrow \texttt{silnia(1)}.$$

Wyniki wracają w odwrotnej kolejności: $1, 2, 6$.

```python
import sys

# Używamy obu funkcji silni z wcześniejszego slajdu.
n = sys.getrecursionlimit() + 50
print("Limit:", sys.getrecursionlimit())
try:
    silnia_rekurencja(n)
except RecursionError:
    print("RecursionError: zbyt wiele aktywnych wywołań")

wynik = silnia_iteracja(n)
print("Wersja iteracyjna zakończyła obliczenia:", wynik > 0)
```

RecursionError oznacza przekroczenie limitu głębokości, a nie wyczerpanie całej pamięci RAM (`MemoryError`). Nie zwiększamy limitu do tego pokazu.

Zastosowanie rekurencji

Przykłady problemów rekurencyjnych:

- **Przeszukiwanie drzewa** – przetwarzamy węzeł i jego poddrzewa. Wersja iteracyjna może przechowywać oczekujące węzły na jawnym stosie.
- **Analiza wyrażeń** – parser rozpoznaje zagnieżdżone wyrażenia, np. nawiasy wewnątrz nawiasów.
- **Fraktale geometryczne** – kolejne poziomy krzywej Kocha lub trójkąta Sierpińskiego powstają z podobnych, mniejszych części.
- **Wieże Hanoi** – przeniesienie n krążków wymaga rozwiązania problemu dla $n - 1$ krążków.
- **Sortowanie** – quicksort i mergesort dzielą dane na mniejsze części. Istnieją też wersje iteracyjne, np. scalanie od dołu.

Rekurencja w modelach procesów naturalnych

Rekurencja może opisywać strukturę modelu; sam proces fizyczny lub biologiczny nie jest wywołaniem funkcji.

- **Drzewo filogenetyczne** – rekurencyjne przechodzenie drzewa pokrewieństwa ewolucyjnego gatunków.
- **Podziały komórek** – model drzewa potomków: każda komórka może dać początek kolejnym gałęziom. Symulację kończymy np. po ustalonej liczbie pokoleń.
- **Łańcuch rozpadu** – nuklid potomny może również być promieniotwórczy. Model przechodzi do kolejnego nuklidu aż do stabilnego produktu. Nie jest to to samo co reakcja łańcuchowa rozszczepienia.
- **Drzewo zakażeń** – modelujemy kolejne osoby zakażone przez daną osobę, z ustalonym warunkiem zakończenia symulacji.

Sortowanie z wyrażeniem lambda

Wyrażenia lambda w Pythonie to anonimowe funkcje zawierające jedno wyrażenie, które mogą być tworzone bez formalnej definicji za pomocą `def`. Są one szczególnie użyteczne w sytuacjach, gdy potrzebna jest prosta funkcja do jednorazowego użytku.

Składnia:

```
lambda argumenty: wyrażenie
```

- `lambda`: Słowo kluczowe wprowadzające wyrażenie lambda.
- `argumenty`: Lista parametrów wejściowych, może być pusta.
- `wyrażenie`: Jedno wyrażenie, którego wynik zostanie zwrócony.

Przykłady:

Proste wyrażenie lambda:

```
dodaj = lambda x, y: x + y
print(dodaj(2, 3))  # Wyświetli: 5
```

Sortowanie listy krotek:

```
lista = [(1, 'b'), (2, 'a'), (3, 'c')]
lista.sort(key=lambda x: x[1])
print(lista)  # Wyświetli: [(2, 'a'), (1, 'b'), (3, 'c')]
```

Wyrażenie lambda

Przykłady:
Użycie z funkcją `map`:

```python
liczby = [1, 2, 3, 4]
kwadraty = list(map(lambda x: x ** 2, liczby))
print(kwadraty)  # Wyświetli: [1, 4, 9, 16]
```

Zastosowania

- **Funkcje wyższego rzędu**: Wyrażenia lambda są często używane jako argumenty funkcji takich jak `map`, `filter`, `reduce`.
- **Sortowanie**: Używane jako klucz sortowania w funkcjach sortujących.
- **Callbacki**: W interfejsach API wymagających funkcji zwrotnych.

Ograniczenia

- **Jedno wyrażenie**: Nie zawiera instrukcji, np. pętli `for` ani `return`. Może zawierać wyrażenie warunkowe, np. `lambda x: x if x >= 0 else -x`.
- **Anonimowość**: Przypisanie do zmiennej nie zmienia atrybutu `__name__`, który pozostaje równy `<lambda>`.
- **Czytelność**: Nadużywanie wyrażeń lambda może prowadzić do mniej czytelnego kodu.

Porównanie z funkcjami zdefiniowanymi za pomocą `def`

Funkcja standardowa:

```
1 def dodaj(x, y):
2     return x + y
```

Wyrażenie lambda:

```
1 dodaj = lambda x, y: x + y
```

Obie funkcje działają identycznie, ale funkcja zdefiniowana za pomocą `def` może zawierać bardziej skomplikowaną logikę i jest zazwyczaj bardziej czytelna.

Dobre praktyki

- Używaj wyrażeń lambda dla prostych funkcji jednorazowego użytku.
- Dla bardziej złożonych funkcji używaj standardowej definicji za pomocą `def`.
- Unikaj zagnieżdżania wyrażeń lambda, aby nie obniżać czytelności kodu.

Funkcje `map` i `filter`

`map` i `filter` są wbudowane. Funkcja `reduce` wymaga importu z modułu `functools`.
Funkcja `map`
`map(function, iterable, ...)` zwraca iterator, który stosuje podaną funkcję do każdego elementu iterowalnego obiektu. **Przykład:**

```
liczby = [1, 2, 3, 4]
kwadraty = list(map(lambda x: x ** 2, liczby))
print(kwadraty)  # Wyświetli: [1, 4, 9, 16]
```

Wiele obiektów iterowalnych (domyślnie do wyczerpania najkrótszego):

```
a = [1, 2, 3]
b = [4, 5, 6]
suma = list(map(lambda x, y: x + y, a, b))
print(suma)  # Wyświetli: [5, 7, 9]
```

Zastosowanie:

- Przekształcanie danych.
- Zastosowanie funkcji do wielu iterowalnych obiektów równocześnie.

Funkcje `map` i `filter`

Funkcja `filter`

`filter(function, iterable)` zwraca iterator zawierający elementy, dla których funkcja zwróciła wartość prawdziwą. **Przykład:**

```
liczby = [1, 2, 3, 4, 5, 6]
parzyste = list(filter(lambda x: x % 2 == 0, liczby))
print(parzyste)  # Wyświetli: [2, 4, 6]
```

Zastosowanie:

- Filtrowanie danych na podstawie warunku.
- Usuwanie elementów niespełniających kryteriów.

map, filter i wyrażenia listowe

Porównanie z wyrażeniami listowymi

`map` i `filter` zwracają leniwe iteratory: elementy powstają podczas odczytu, a wyczerpanego iteratora nie odczytamy ponownie. Wyrażenie listowe od razu tworzy listę. **Przykład `map` vs wyrażenie listowe:**

```
liczby = [1, 2, 3, 4, 5, 6]
# Użycie map
kwadraty = list(map(lambda x: x ** 2, liczby))

# Wyrażenie listowe
kwadraty = [x ** 2 for x in liczby]
```

Przykład `filter` vs wyrażenie listowe:

```
# Użycie filter
parzyste = list(filter(lambda x: x % 2 == 0, liczby))

# Wyrażenie listowe
parzyste = [x for x in liczby if x % 2 == 0]
```

Sekcja 2

Podstawy obsługi plików

Otwarcie i Zamknięcie Pliku

Funkcja `open()`
Aby rozpocząć pracę z plikiem, należy go otworzyć. W Pythonie służy do tego funkcja `open()`, która zwraca obiekt pliku umożliwiający wykonywanie operacji I/O (wejścia/wyjścia).

```
1 file = open('nazwa_pliku.txt', 'r', encoding='utf-8')
```

- Pierwszy argument (`'nazwa_pliku.txt'`): Ścieżka do pliku, który chcemy otworzyć. Może to być nazwa pliku w bieżącym katalogu lub pełna ścieżka do pliku.
- Drugi argument (`'r'`): Tryb otwarcia pliku, określający, w jaki sposób będziemy z nim pracować (np. odczyt, zapis).

Zamknięcie pliku
Po zakończeniu operacji na pliku należy go zamknąć, aby zwolnić zasoby systemowe i upewnić się, że wszystkie dane zostały zapisane.

```
1 file.close()
```

Dlaczego to ważne? Niezamknięcie pliku może prowadzić do wyczerpania limitu otwartych plików w systemie lub utraty danych.

Tryby Otwarcia Pliku

Podczas otwierania pliku możemy określić różne tryby, które definiują sposób interakcji z plikiem.

- `'r'` – **odczyt (read)**: Domyślny tryb. Otwiera plik do odczytu. Plik musi istnieć, w przeciwnym razie zostanie zgłoszony błąd `FileNotFoundError`.
- `'w'` – **zapis (write)**: Otwiera plik do zapisu. Jeśli plik nie istnieje, zostanie utworzony. Jeśli istnieje, jego zawartość jest usuwana już przy otwarciu.
- `'a'` – **dopisywanie (append)**: Otwiera plik do dopisywania na końcu. Jeśli plik nie istnieje, zostanie utworzony. Dane są dodawane na końcu pliku bez nadpisywania istniejącej zawartości.
- `'x'` – **utworzenie nowego pliku**: Jeśli plik już istnieje, zgłasza `FileExistsError` i nie usuwa jego zawartości.
- `'r+'` – **odczyt i zapis**: Otwiera plik do odczytu i zapisu. Plik musi istnieć.
- **Tryb binarny** (`'b'`): Np. `'rb'` lub `'wb'`. Służy do odczytu i zapisu bajtów, np. w plikach obrazów.

Warianty: `b` oznacza bajty, bez `encoding`; tryb tekstowy zwraca napisy.

```
# Warianty argumentów (wybieramy właściwy tryb):
# open('obrazek.jpg', 'rb')   -- odczyt bajtów
# open('dane.bin', 'wb')      -- zapis bajtów
# open('notatki.txt', 'r+')  -- odczyt i zapis
```

Odczyt tekstu: read i readline

Odczyt zaczyna się od bieżącej pozycji. Każda operacja przesuwa ją dalej.

Metoda `read()`

```
content = file.read()  # Pozostały tekst do końca pliku
file.seek(0)           # Powrót na początek pliku
fragment = file.read(100)  # Do 100 kolejnych znaków
```

Po odczycie do końca kolejne `read()` zwraca `''`. Aby wrócić na początek, można użyć `file.seek(0)` lub ponownie otworzyć plik.

Metoda `readline()`

```
line = file.readline()  # Jeden kolejny wiersz
```

Zachowuje końcowy znak `'\n'`, jeśli występuje. Pusta linia to `'\n'`; koniec pliku to `''`.

Pamięć: `read()` bez argumentu wczytuje cały pozostały tekst. Przy dużych plikach warto przetwarzać kolejne wiersze.

Operacje na Plikach Tekstowych

- **Metoda `readlines()`**

Czyta pozostałe linie od bieżącej pozycji i zwraca ich listę; każdy element to jeden wiersz.

```
lines = file.readlines()
```

Uwaga: Podobnie jak `read()`, może być nieefektywne przy dużych plikach.

- **Iterowanie po Pliku**

Plik w Pythonie jest iterowalny, więc możemy iterować po nim bezpośrednio w pętli `for`.

```
for line in file:
    print(line, end='')
```

- Jest to efektywny sposób na czytanie pliku linia po linii.
- **Zaleta:** Nie ładuje całego pliku do pamięci.

Zapisywanie Danych do Pliku

Po otwarciu pliku w trybie zapisu (`'w'`, `'a'`, `'r+'`, itp.) możemy zapisywać dane.

- **Metoda `write()`**

Zapisuje ciąg znaków do pliku.

```
1 file.write('Przykładowy tekst.\n')
```

Uwaga: Metoda `write()` nie dodaje automatycznie znaku nowej linii. Jeśli chcemy rozpocząć nową linię, musimy dodać `'\n'` na końcu.

- **Metoda `writelines()`**

Zapisuje elementy iterowalnej kolekcji napisów do pliku (np. listy).

```
1 lines = ['Linia 1\n', 'Linia 2\n']
2 file.writelines(lines)
```

Uwaga: Metoda `writelines()` nie dodaje znaków nowej linii między elementami listy. Musimy je uwzględnić w samych ciągach znaków.

Menedżer kontekstu (`with`)

Użycie instrukcji `with` jest zalecaną praktyką podczas pracy z plikami. Zapewnia ona automatyczne zamknięcie pliku po zakończeniu bloku kodu, nawet jeśli wystąpi wyjątek.

```
with open('nazwa_pliku.txt', 'r', encoding='utf-8') as file:
    content = file.read()
# Po wyjściu z bloku 'with' plik jest automatycznie zamknięty
```

Zalety:

- Poprawia czytelność kodu.
- Unika konieczności ręcznego wywoływania `file.close()`.
- Zwiększa bezpieczeństwo i niezawodność aplikacji.

Czytanie wierszami z automatycznym zamknięciem:

```
with open('dane.txt', 'r', encoding='utf-8') as file:
    for line in file:
        print(line, end='')
```

Obsługa błędów podczas pracy z plikami

`with` zamyka otwarty plik także przy wyjątku. Przechwytywanie błędów realizujemy osobno, za pomocą `try-except`.

```
try:
    with open('dane.txt', 'r', encoding='utf-8') as file:
        content = file.read()
except FileNotFoundError:
    print('Plik nie został znaleziony.')
except PermissionError:
    print('Brak uprawnień do otwarcia pliku.')
except OSError as e:
    print(f'Inny błąd wejścia/wyjścia: {e}')
except UnicodeDecodeError:
    print('Zawartość nie jest poprawnym tekstem UTF-8.')
else:
    print(content)
```

Kolejność ma znaczenie: Konkretne wyjątki umieszczamy przed ogólnym `OSError`. Gdy `open()` się nie powiedzie, nie ma pliku do zamknięcia; nie wywołujemy wtedy `file.close()` w `finally`.

Kodowanie Znaków

Przy pracy z plikami tekstowymi, zwłaszcza zawierającymi znaki spoza ASCII, ważne jest określenie kodowania.

```
with open('nazwa_pliku.txt', 'r', encoding='utf-8') as file:
    content = file.read()
```

- `encoding='utf-8'`: Określa, że plik jest kodowany w UTF-8, co jest standardem dla większości języków.

Popularne Kodowania

- `'utf-8'`: Obsługuje wszystkie znaki Unicode. Zalecane do ogólnego użytku.
- `'latin-1'` (ISO-8859-1): Kodowanie dla Europy Zachodniej. Każdy bajt jest ważnym znakiem.
- `'cp1250'`: Kodowanie Windows dla Europy Środkowej.

Właściwości Obiektu Pliku

Obiekt pliku w Pythonie posiada szereg atrybutów, które dostarczają informacji o stanie pliku.

- `file.name`: Nazwa pliku lub jego ścieżka.
- `file.mode`: Tryb, w jakim plik został otwarty.
- `file.closed`: Wartość logiczna wskazująca, czy plik jest zamknięty.

```
with open('dane.txt', 'r', encoding='utf-8') as file:
    print(f'Nazwa pliku: {file.name}')
    print(f'Tryb otwarcia: {file.mode}')
print(f'Czy plik jest zamknięty? {file.closed}')
```

Sekcja 3

Przykłady

Zmienna globalna i UnboundLocalError

Przypisanie w funkcji czyni nazwę lokalną, o ile nie zadeklarowano inaczej. `counter += 1` najpierw odczytuje jej wartość; tu odczyt następuje przed lokalnym przypisaniem.

Odczyt niezdefiniowanej nazwy lokalnej

```
counter = 0

def increment():
    counter += 1

increment()  # UnboundLocalError
```

Jawna zmiana nazwy globalnej

```
counter = 0

def increment():
    global counter
    counter += 1

increment()
print(counter)  # 1
```

Samo `counter = 1` bez wcześniejszego odczytu utworzyłoby nazwę lokalną bez tego błędu. Często czytelniej przekazać stan jako argument i zwrócić nową wartość.

Automatyczne zamykanie pliku

Brak zamknięcia pliku po zakończeniu operacji może prowadzić do wyczerpania dostępnych deskryptorów plików lub utraty danych, ponieważ bufor zapisu może nie zostać opróżniony.

```python
def read_file_contents(filename):
    file = open(filename, 'r', encoding='utf-8')
    content = file.read()
    return content  # Plik nie został zamknięty

# Wywołanie funkcji
data = read_file_contents('dane.txt')
```

Automatyczne zamykanie pliku

Brak zamknięcia pliku po zakończeniu operacji może prowadzić do wyczerpania dostępnych deskryptorów plików lub utraty danych, ponieważ bufor zapisu może nie zostać opróżniony.

```python
def read_file_contents(filename):
    file = open(filename, 'r', encoding='utf-8')
    content = file.read()
    return content  # Plik nie został zamknięty

# Wywołanie funkcji
data = read_file_contents('dane.txt')
```

```python
def read_file_contents(filename):
    with open(filename, 'r', encoding='utf-8') as file:
        content = file.read()
    return content  # Plik jest już zamknięty
```

Czytanie pliku linia po linii

Porównujemy odczyt całej listy linii z przetwarzaniem strumieniowym. Plik zawiera jedną liczbę w każdym niepustym wierszu.

```
with open('pomiary.txt', 'r', encoding='utf-8') as file:
    lines = file.readlines()  # Lista wszystkich linii w pamięci
    suma = sum(float(line) for line in lines if line.strip())
print(suma)
```

Czytanie pliku linia po linii

Porównujemy odczyt całej listy linii z przetwarzaniem strumieniowym. Plik zawiera jedną liczbę w każdym niepustym wierszu.

```
with open('pomiary.txt', 'r', encoding='utf-8') as file:
    lines = file.readlines()  # Lista wszystkich linii w pamięci
    suma = sum(float(line) for line in lines if line.strip())
print(suma)
```

```
suma = 0.0
with open('pomiary.txt', 'r', encoding='utf-8') as file:
    for line in file:
        if line.strip():
            suma += float(line)
print(suma)
```

Druga wersja nie przechowuje wszystkich linii. Zużycie pamięci zależy m.in. od długości pojedynczej linii i buforowania; nie od samej liczby wierszy.

Ochrona pliku przed nadpisaniem

Tryb `'w'` usuwa dotychczasową zawartość już przy otwarciu pliku:

```
1 with open('wyniki.txt', 'w', encoding='utf-8') as file:
2     file.write('Nowa zawartość.\n')
```

Ochrona pliku przed nadpisaniem

Tryb `'w'` usuwa dotychczasową zawartość już przy otwarciu pliku:

```
with open('wyniki.txt', 'w', encoding='utf-8') as file:
    file.write('Nowa zawartość.\n')
```

Tryb `'x'` pozwala utworzyć wyłącznie nowy plik:

```
try:
    with open('wyniki.txt', 'x', encoding='utf-8') as file:
        file.write('Nowa zawartość.\n')
except FileExistsError:
    print('Plik już istnieje; zachowano jego zawartość.')
```

Do dopisania danych na końcu istniejącego pliku służy `'a'`.

Sekcja 4

Zadanie dodatkowe

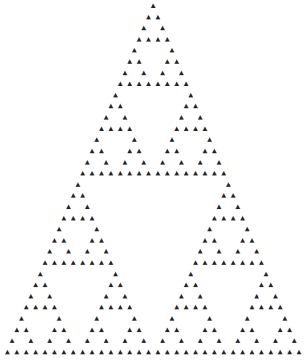
Zadanie dodatkowe

Zadanie (2 punkty)

Napisz program wyświetlający trójkąt Sierpińskiego w trybie tekstowym. Wykorzystaj trójkąt Pascala: drukuj znak w miejscach liczb nieparzystych, a spację w miejscach liczb parzystych.

Wskazówki:

- Nieparzyste elementy trójkąta Pascala tworzą wzór trójkąta Sierpińskiego.
- Znak ▲ uzyskasz przez `chr(9650)`.



Przykładowy wynik działania programu.

Kod programu proszę przesłać na adres jacek.gatlik@agh.edu.pl.